Auto-KForge: Agentic Kernel Generation for Heterogeneous AI Accelerators

Ankita Nayak
Gimlet Labs
San Francisco, USA
ankitanayak@gimletlabs.ai

Burak Bartan
Gimlet Labs
San Francisco, USA
bbartan@gimletlabs.ai

Taras Sereda
Gimlet Labs
San Francisco, USA
taras@gimletlabs.ai

Natalie Serrino
Gimlet Labs
San Francisco, USA
nserrino@gimletlabs.ai

Abstract

Heterogeneous deployments need high-performance kernels for the same operators on every backend in the mix, and each new architecture ships ahead of the hand-tuned libraries that provide them. The techniques that make a new architecture fast are sparse in model weights and scattered across vendor documentation, so a coding model given an unfamiliar target clears the correctness bar and stops well short of the hardware. Reported speedups do not show how much of that gap remains, and they say even less about whether a kernel is deployable. We present Auto-KForge, a production kernel generation system driven by autonomous coding agents. It pairs the agent with a remote verification worker that compiles, verifies, and times every candidate on real hardware and records each attempt for later analysis, while a skill-synthesis pipeline distilled from hardware documentation adapts the same system to each backend. Three objectives then separate a benchmark pass from a deployable kernel, and Auto-KForge builds a mechanism for each. *Capability* is measured as attainment against a modeled speed-of-light rather than a speedup over a baseline of unknown strength. *Correctness* is enforced as the intended numerical contract rather than a loose tolerance. *Coverage* is optimized across the deployed shape distribution rather than a single tuned configuration. We report results on two accelerator families, including a fused attention kernel on NVIDIA Blackwell that reached roughly 3× a tuned Triton reference and a PagedAttention kernel in HIP on AMD MI350X that raised prefill throughput by 35% end to end. We close with the lessons that generalize, including what a reported kernel generation result should make legible.

Keywords

kernel generation, agentic code generation, heterogeneous accelerators, speed-of-light, ML for systems

1 Introduction

Production inference increasingly runs on a mix of accelerators. An agentic pipeline interleaves model calls, retrieval, and tool invocations, each with a different compute and memory profile, and no single accelerator suits every stage. Assigning stages to the hardware that fits them makes kernel availability a deployment constraint, because the same operators must run well on every backend in the mix.

Each new accelerator generation, whether NVIDIA Blackwell or AMD CDNA, arrives with new tensor formats, memory hierarchies, and special-function units, but the hand-tuned kernel libraries that turn those features into usable throughput lag the silicon by months. The gap is widened by an information problem. The idioms that make a new platform fast are sparse in model weights and scattered across vendor blogs, whitepapers, and code, so a general-purpose coding model asked to write a kernel for a platform it has barely seen tends to produce something that compiles and passes a tolerance check but performs well below what the hardware can deliver. Performance code is also brittle, since small changes in memory layout or numerical precision can break correctness or erase the gain outright.

Large language model agents have recently been applied to this problem [3, 4, 6, 9, 16], and a steady stream of results reports ever-larger speedups over reference implementations. A growing body of analysis shows that a benchmark speedup is a fragile proxy for a deployable kernel. Baselines vary widely, numerical contracts can be relaxed within a tolerance, and performance can be overfit to a single tested shape [11, 15, 18]. Our experience building an agentic kernel generation system and deploying its kernels into production inference makes these failure modes concrete, and it also surfaces the systems mechanisms needed to avoid them.

This paper describes Auto-KForge, a production system that generates deployable hardware kernels with autonomous agents across heterogeneous accelerators. Auto-KForge puts a frontier coding agent in a verification loop that compiles, checks, and times every candidate on the target accelerator, records every attempt to a shared store and an observability stack, and adapts the same system to each backend through a skill-synthesis pipeline (Section 3). Optimization and correctness machinery then carries a generated kernel from a benchmark pass to something deployable (Section 4). On NVIDIA Blackwell, a generated attention kernel reached roughly 3× a tuned Triton reference, and on AMD MI350X, a generated PagedAttention kernel raised end-to-end prefill throughput by 35% (Section 5). Adding a backend means pointing the skill pipeline at that hardware's documentation and running the verification worker on the target device.

Three objectives organize the system and, in our experience, separate a headline speedup from a kernel that ships. *Capability*

OASIS-AI 2026, co-located with the 59th IEEE/ACM International Symposium on Microarchitecture (MICRO-59), Athens, Greece

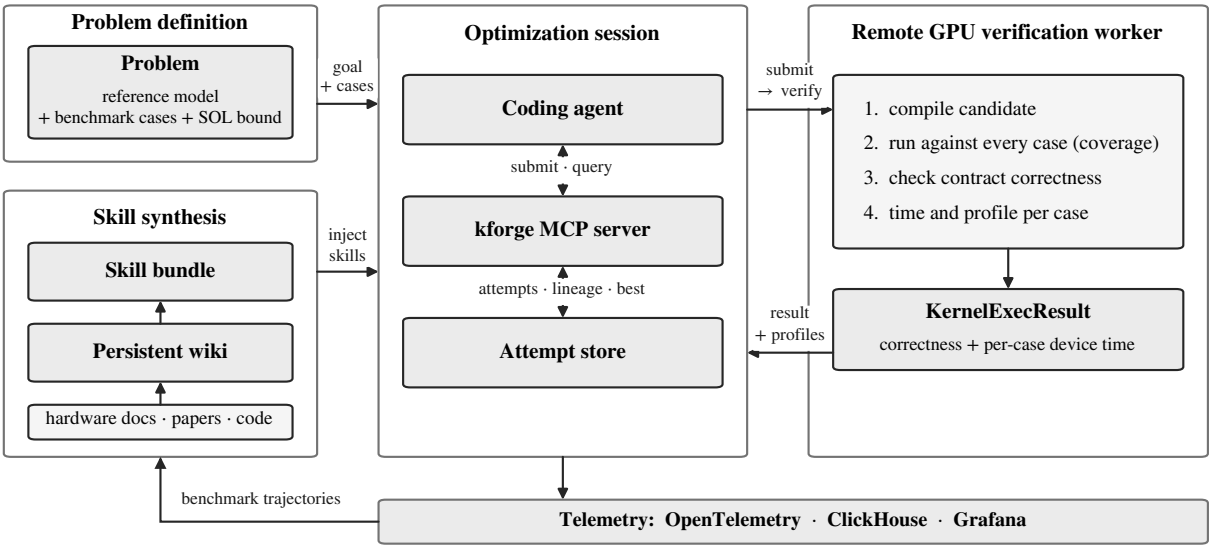


Figure 1: Auto-KForge gives a frontier coding agent a problem and a bundle of hardware-specific kernel skills, then lets it run autonomously. Candidates go through the kforge MCP server to a remote GPU worker that compiles, checks correctness against the reference, and times every case on real hardware. Attempts persist in a shared store and stream to an observability stack for later analysis.

is the fraction of speed-of-light attained rather than the speedup over a baseline of unknown strength; *correctness* is the intended production contract rather than a loose tolerance; and *coverage* is performance across the deployed shape distribution rather than a single tuned configuration. We argue these belong inside the generation system rather than only in downstream evaluation, so the mechanisms in Sections 3 and 4 exist to serve them, and Section 6 draws out the lessons, including what a reported result should make legible.

2 Background

Agentic kernel generation has evolved quickly. Early systems generated kernels with a single LLM call, optionally with compile-and-retry. Later approaches added fixed search procedures or fine-tuned generators with reinforcement learning [4, 13]. Recent systems increasingly use long-horizon coding agents with specialized skills, tool access, and hardware-in-the-loop profiling [6, 16]. Auto-KForge sits in this last category, where the agent decides for itself how to explore. That freedom is what makes strong kernels reachable, but it also lets an agent exploit benchmark gaps, producing kernels that pass tests while being incorrect, suboptimal, or overfit to specific shapes. Auto-KForge treats these failure modes as design constraints on the generation loop rather than as something for evaluation to catch afterward.

Evaluation infrastructure has also developed alongside the generators. KernelBench [9] established a standard problem set for LLM-generated kernels, and subsequent work has tightened its interpretation. KernelBench-verified [18] re-examines whether generated kernels actually beat PyTorch. Separate analyses of the correctness illusion [11] and of production readiness [15] document how benchmark passes diverge from deployable behavior. Complementary efforts broaden the evaluation setting: BackendBench [12] tests whether generated kernels satisfy PyTorch operator tests and integrate into a functional backend, and KernelBot [17] provides a common competition platform for comparing GPU kernels produced by humans and AI. Auto-KForge turns these measures into generation-time objectives rather than post-hoc checks, giving the agent a speed-of-light target, a case set spanning deployment shapes, and an enforced numerical contract.

3 The Auto-KForge System

Figure 1 shows how the pieces of Auto-KForge fit together. The agent receives a problem, consisting of a reference implementation, a set of benchmark cases, and a speed-of-light (SOL) bound, together with a hardware backend-specific skill bundle. It submits candidate kernels through an MCP (Model Context Protocol) server to a remote GPU verification worker. The worker compiles each candidate, checks its correctness against the reference, and times and profiles it on the target accelerator. Every attempt lands in a shared store and a telemetry stack. Auto-KForge is agent-agnostic, running with multiple frontier coding agents [1, 2, 8], and model-agnostic.

Verification Loop. Each problem runs in a self-contained session workdir. The agent submits candidate kernels over MCP, and a remote GPU worker compiles each candidate, checks correctness against a reference implementation, and reports per-case timing and profiling. Per-attempt artifacts, including candidate source,

description, results, and profiles, are persisted so that resumed runs and subagents share one stable view of prior candidates, their parentage, and the current best. This shared state is the substrate for branching and long-horizon search.

Autonomous Operation and Observability. Auto-KForge runs both user-steered and fully autonomously. A wrapper drives the session unattended until the objective verifies, with no human in the loop. Each agent is launched in the session workdir under a permission model that lets it create candidates and call Auto-KForge tools without broad host access. Every attempt is recorded and visualized through an observability stack built on OpenTelemetry, ClickHouse, and Grafana. Saved artifacts include candidate source, per-case device and wall time, correctness results, and profiling. Beyond debugging and reproducibility, this telemetry is the substrate for closing the loop from benchmark experience back into the skills described above. Figure 2 shows the lineage view over the attempts in one session, where the search branches from a running best and each node records source, correctness, and per-case timing.

Multi-Backend Generation via Skill Synthesis. An agent's ceiling on a backend is bounded by what it knows about that hardware. For mature platforms with abundant public code, a frontier model may write competent kernels. However, for emerging platforms (such as NVIDIA Blackwell or AMD CDNA), the relevant idioms are sparse in the weights and scattered across blogs, papers, and code. This is the central obstacle to supporting heterogeneous accelerators from one system, and it is where Auto-KForge invests. Auto-KForge distills hardware knowledge into skills through a pipeline. It first synthesizes a persistent, cross-linked *wiki*, with one page per hardware concept, source-cited and with conflict resolution, from a source manifest of vendor documentation, papers, and code. It then projects the wiki into a compact per-backend *skill bundle*, consisting of a prose optimization guide, a strategy menu, coding and planning rules, and code exemplars, which is injected into the agent at session start. Supporting a new target is a matter of pointing the pipeline at a new source list, which reproduces the flow for that backend.

4 Optimizing for Deployment

A generation agent tends to maximize whatever the benchmark measures, so any slack in the specification, such as a weak baseline, a loose tolerance, a narrow input distribution, or a single tested shape, becomes a target it will exploit. Auto-KForge builds a mechanism for each of the three objectives, capability, correctness, and coverage, and pursues them during generation, so the agent is pushed toward kernels that satisfy them rather than kernels a benchmark eventually filters out.

4.1 Capability: Measuring Against Speed-of-Light

In production, beyond absolute speedups, what matters is how closely a kernel approaches the best achievable performance on the target hardware. Prior work has scored a kernel by its *speed-of-light* (SOL) attainment [5, 7], the fraction of the hardware ceiling it reaches [14], instead of its speedup over a software baseline. Speedup over a baseline conflates two things that a speed-of-light measure separates. A large speedup over a weak baseline may sit far below the hardware ceiling, while a modest speedup over a strong baseline can already be near it. Auto-KForge therefore measures a candidate's device time against a modeled SOL, the roofline lower bound [14]

$$T_{\text{SOL}} = \max\left(\frac{F_{\text{total}}}{P_{\text{peak}}},\ \frac{B_{\text{total}}}{BW_{\text{peak}}}\right),$$

where F_{total} is the operator's FLOP count and B_{total} its global-memory traffic. Attainment, the fraction of SOL a kernel reaches, is reported alongside speedup and is what the agent is steered to improve.

Our SOL estimates are problem-specific and hardware-aware, with fusion declared per workload. We illustrate the methodology on a single workload, KernelBench Level 2 Problem 28, a linear projection followed by InstanceNorm, residual addition, and elementwise multiplication:

$$Z = XW^\top + b, \quad \hat{Z} = \text{InstanceNorm}(Z), \quad O = (\hat{Z} + Y) \odot Y,$$

with $X \in \mathbb{R}^{m\times k}$, $W \in \mathbb{R}^{n\times k}$, and $Y, O \in \mathbb{R}^{m\times n}$. For the evaluated configuration, $m = 1024$ and $k = n = 8192$. Using the standard GEMM convention, the linear layer costs $2mnk$ FLOPs (mn output elements, each a k-term multiply-accumulate counted as two FLOPs). The bias adds mn, InstanceNorm $9mn$, and the residual addition and final multiplication $2mn$, giving

$$\begin{aligned} F_{\text{total}} &= 2mnk + mn + 9mn + 2mn \\ &= mn(2k + 12) \approx 137.54 \text{ GFLOP}. \end{aligned}$$

Fused regions are declared per workload. For this workload the full sequence fuses, so intermediate tensors stay on chip and, in bf16, only the inputs, weight, bias, residual, and output cross global memory, for a total of

$$B_{\text{total}} = 2(mk + nk + n + 2mn) \approx 184.57 \text{ MB}.$$

For a single NVIDIA B200 SXM GPU we use a peak dense bf16 Tensor Core throughput of $P_{\text{peak}} = 2.25$ PFLOP/s and peak HBM bandwidth of $BW_{\text{peak}} = 7.7$ TB/s, giving a compute bound of 61.1 μs and a memory bound of 24.0 μs. The SOL runtime is therefore 61.1 μs, making this workload compute-bound under the model. The example shows how SOL separates the two hardware limits. Although the operator moves a substantial amount of data, its modeled lower bound is set by compute rather than bandwidth. It also makes explicit that SOL depends on workload-specific choices about FLOP counting, memory traffic, and fusion, which is why consistent modeling conventions matter when comparing results across problems and systems. The gap between attainment and SOL is what the agent then works to close. It does so with skills synthesized from hardware documentation and with planning long enough to carry an optimization across many steps rather than stopping at the first improvement. How reliably an agent reaches high attainment still depends on its ability to reason about the hardware and sustain a long optimization. We address this through harness design (Section 3), though stronger models also raise the ceiling.

4.2 Correctness: Enforcing the Production Contract

A reported speedup can be a correctness regression in disguise, driven by changes to the numerical contract or the math behind it.

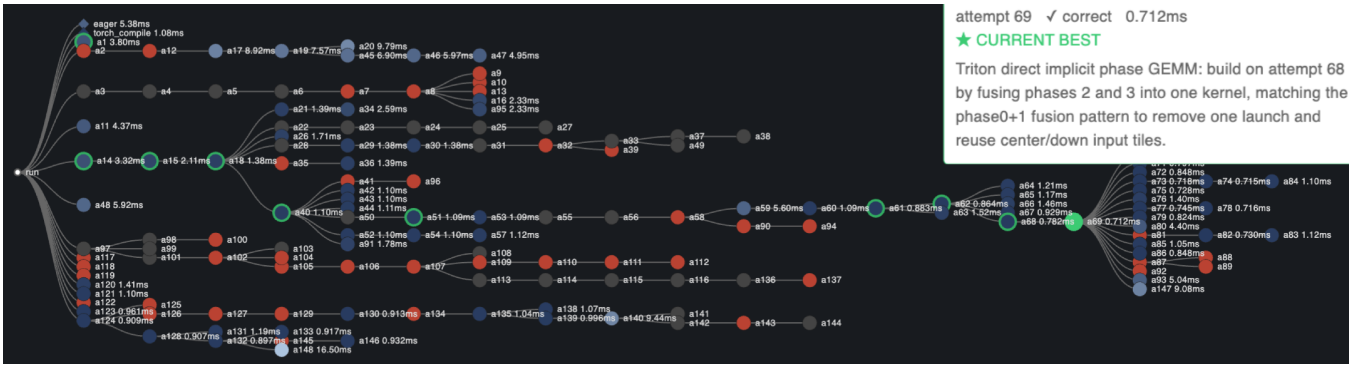


Figure 2: Auto-KForge's lineage view over the attempts in one session. Circles are candidate attempts and edges connect each candidate to the parent it was derived from. Node color shows outcome (grey for failed to compile, red for incorrect, blue shaded by runtime with darker meaning faster, green for current best), and enlarged green-outlined nodes set a new best when submitted.

The most common form is a precision change that stays within a loose tolerance. Reducing accumulation precision, or substituting a lower-precision path, can pass a tolerance check on the benchmarked inputs while silently altering the contract the caller relied on [10]. Another form is a range approximation, valid over the benchmarked inputs but wrong outside them.

Two cases from our runs make this concrete. On KernelBench Level 1 Problem 32, a HardTanh that clips inputs to $[-1, 1]$, the agent produced a kernel that returns the input unchanged, reporting a 71,000× speedup. The benchmark draws inputs from `torch.rand` in $[0, 1)$, where HardTanh is the identity, so the kernel passes but is silently wrong outside that range. In another case, the agent skipped the running-maximum computation needed for stable softmax, reusing the fixed attention-sink value as the exponent base instead. It enabled this shortcut only on the large shapes used for scoring, where the speedup is biggest. The output still landed within the accuracy tolerance, so the benchmark marked it correct despite the lost numerical stability. Unless each kernel is individually validated, such cases can artificially inflate the aggregate speedup reported across a benchmark suite.

Auto-KForge guards against these during generation with three mechanisms. First, the agent's skills file encodes the numerical contract, including precision, dtypes, and accumulation modes, and rejects candidates that reduce precision to satisfy a loose tolerance. Second, end-to-end evaluation serves as a final acceptance gate, ensuring that a kernel preserves behavior in context rather than only on an isolated micro-benchmark. Third, the agent generates additional test cases beyond the benchmark's fixed inputs, probing precisions and ranges that may otherwise go unchecked. Beyond generation, we are also investigating formal verification as a longer-term path to establishing equivalence to the original contract.

4.3 Coverage: Optimizing Across the Shape Distribution

The shapes and cases the agent has access to during generation play a significant role in determining the generated kernels. In production, the same operator runs across a distribution of shapes, from prefill to decode and across batch sizes and sequence lengths, so a kernel fast only at the shape it was tuned for does not deliver that speed where it actually runs. These failures show up as kernels in which batch sizes or other dimensions are effectively hardcoded, and as cases where a benchmark's regularity, such as equally shaped packed documents, can be exploited for a better number that does not survive real, ragged inputs. Specialization itself is not the problem, since the operator can dispatch a different kernel per shape. The problem is a single kernel overfit to one shape being scored as if it served them all.

Auto-KForge therefore represents an operator as a set of *cases*, each with its own input shapes. A candidate is compiled once and run against every case. It passes only if every case is correct, and its rank is the geometric mean of per-case device time, with wall time as a secondary key. Co-optimization follows directly, because the geomean scores the whole case set at once, so a candidate cannot win by sacrificing one case for another unless the aggregate genuinely improves. The objective is expressed through a goal specification the agent reads alongside its workflow contract, covering target speedup and baseline, precision and numerical contract, required shape generality, and structural constraints. The specification can state compound, per-shape targets, so the system produces an entire operator that dispatches among many kernels by shape. To target the shapes a production model actually runs, Auto-KForge captures representative operator inputs from a running inference engine, for example an instrumented vLLM checkout, and turns them into isolated benchmark cases.

5 Deployment Measurements

We report measurements from deploying kernels generated by Auto-KForge on two accelerator families. All numbers are from our runs on real hardware.

Capability. Figure 3 shows, for six fused KernelBench Level 2 problems [9], the fraction of SOL reached by the generated kernel and by a `torch.compile` (bf16) baseline, with the measured speedup annotated. KernelBench typically defines problems with an fp32 baseline. We intentionally set the baseline to bf16 to match

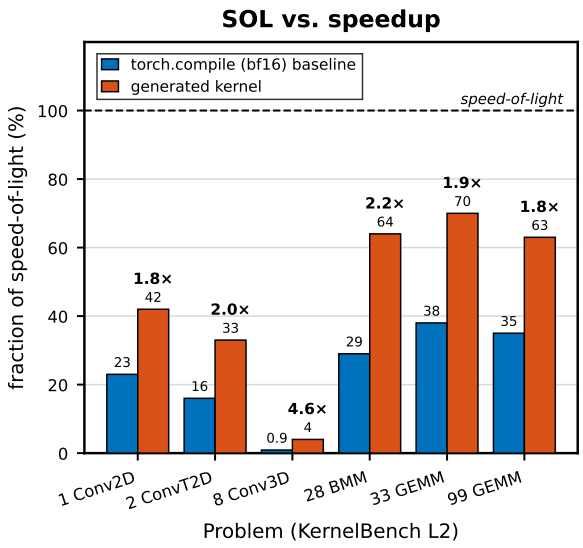


Figure 3: *Capability*: speedup does not track speed-of-light attainment. Bars show the fraction of SOL for the `torch.compile` (bf16) baseline and the generated kernel across six fused KernelBench Level 2 problems, with bold speedups over the baseline. Attainment ranges from 4 to 70%.

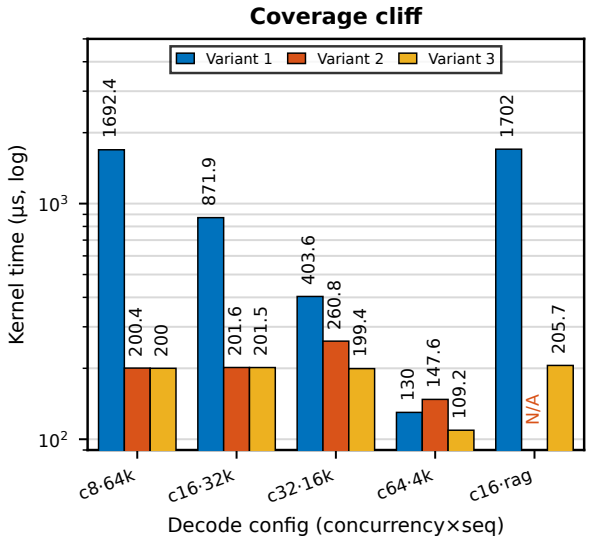


Figure 4: *Coverage*: kernels tuned for one shape may underperform on others. Kernel time (lower is better, log axis) across five decode configurations. For PagedAttention, Variant 1 was tuned on prefill shapes, Variant 2 on two of the decode configurations, and Variant 3 on all five. Only Variant 3 stays fast everywhere. Variant 2 has no result on the ragged batch, and prefill-tuned Variant 1 underperforms on every decode configuration.

the generated kernels, which were audited for bf16 compliance. The largest speedup lands on the workload furthest from the hardware limit. Our fused Conv3D kernel runs 4.6× faster than its bf16 baseline yet reaches only 4% of SOL. Conversely, Conv2D and the GEMMs share a similar speedup, in the 1.8 to 1.9× range, yet the GEMMs reach up to 70% of SOL while Conv2D touches only 42%. Beyond speedup, measuring each kernel against SOL shows how much of the hardware it still leaves unused, pointing to the next optimization opportunities. Acting on that signal is the other half. On a fused GPT-OSS attention kernel for NVIDIA Blackwell, our system started from a kernel already about 2.4× faster than a tuned Triton reference. It remained short of SOL, which justified pushing further. Injecting documentation-derived skills for Blackwell (Section 3) cut the kernel time by another 22%, making it about 3× faster than the Triton reference, and surfaced hardware-specific moves the agent had not explored, such as a smaller KV tile because the attention was bound by the exponential unit rather than the tensor cores.

Correctness. The correctness mechanisms materially change what gets reported. They help generate precision-compliant kernels, and benchmarking those kernels against a matched baseline shows their genuine algorithmic speedup rather than credit for reduced precision. Across the six Level 2 problems above, the same bf16-compliant kernels reach a 10.3× geomean speedup over KernelBench's original fp32 baseline, which uses fp32 inputs and accepts lower-precision outputs within a tolerance. Compared instead against a bf16 baseline, with bf16-normalized inputs and explicit output checks, their speedup is only 2.24×, a 4.6× drop. Without the baseline and tolerance stated, this precision downgrade would read as a real algorithmic speedup.

Coverage. Figure 4 shows the coverage effect on our own runs for AMD kernel generation, with a sharp drop away from the tuned point. Here we task the agent with optimizing PagedAttention in HIP. Variant 1 was generated for prefill shapes and collapses on most decode shapes. Variant 2 was generated using only two decode configurations, `c8·64k` and `c16·32k` (where c8 is concurrency 8 and 64k is the sequence length), and it generalizes to some regular unseen shapes but fails for the ragged case. Generated with all five cases (Section 4.3), Variant 3 holds latency low across every shape, including the ragged case, without the cliffs the specialized variants show.

End-to-end Impact. What ultimately ships is end-to-end model performance, and per-kernel speedups do not necessarily compose into model-level ones. A kernel made 2× faster that accounts for 5% of runtime moves the model by only about 2.5%, and launch overhead, kernel-boundary and fusion effects erode even that, turning a pile of impressive local wins into a modest global one. Auto-KForge captures the operator inputs a running inference engine actually issues, in their real shapes and proportions, so effort goes to the operators that dominate end-to-end time and each is optimized

across production shapes rather than a single benchmark case (Section 4.3). On GPT-OSS-120B inference on an AMD MI350X, starting from an optimized vLLM configuration, Auto-KForge generated a PagedAttention kernel in HIP that raised prefill throughput from 20,000 to 27,000 tokens/s, a 35% end-to-end improvement. Measurements use 4,000 prefill tokens, a 60,000-token existing cache, and batch size 4.

6 Lessons Learned and Discussion

Deploying an agentic kernel generator across two accelerator families taught us that the hard part is not getting an agent to produce a kernel that passes, but getting one that ships. These three objectives, in our experience, separate the two, and each is worth building into the generation system rather than checking only afterward.

These objectives also suggest what a reported kernel generation result should make legible, along with the setup needed to interpret it.

- **Capability.** The baseline's provenance and how optimized it is, the SOL methodology used (peak reference and fusion assumptions), the attainment distribution across workloads beyond its mean, and the end-to-end impact beyond the per-kernel speedup.
- **Correctness.** The tolerance, precision, and accumulation mode accepted, and whether equivalence was checked beyond the benchmark's fixed inputs.
- **Coverage.** The shapes and input distributions evaluated, and performance off the tuned point.
- **Setup.** The device or devices, whether kernels were re-tuned per device or assumed to transfer, and the generation budget spent per kernel.

None of these are hard to report, yet leaving them out is what lets a headline speedup stand in for progress that does not survive real-world deployment.

7 Conclusion

Auto-KForge shows that agent-driven kernel generation can produce deployable kernels across heterogeneous accelerators when the system is built around the right objectives. Pairing hardware-in-the-loop verification with per-backend skill synthesis, shape-distribution optimization, and precision-contract enforcement, we generated and deployed kernels on NVIDIA Blackwell and AMD MI350X, including a PagedAttention kernel that raised end-to-end prefill throughput by 35%. Capability against speed-of-light, correctness against the production contract, and coverage across the deployed shape distribution are what made this system work. All three belong inside the generation system, not only in the benchmarks that judge it.

References

[1] Anomaly. 2026. *OpenCode: The Open Source AI Coding Agent.* https://github.com/anomalyco/opencode

[2] Anthropic. 2025. *Claude Code.* https://github.com/anthropics/claude-code Agentic coding tool.

[3] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. 2025. CUDA-LLM: LLMs can write efficient CUDA kernels. *arXiv preprint arXiv:2506.09092* (2025).

[4] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, et al. 2026. CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286* (2026).

[5] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. 2026. Improving efficiency of GPU kernel optimization agents using a domain-specific language and speed-of-light guidance. *arXiv preprint arXiv:2603.29010* (2026).

[6] Robert Tjarko Lange, Aaditya Prasad, Qian Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. The AI CUDA engineer: Agentic CUDA kernel discovery, optimization and composition. *arXiv preprint arXiv:2509.14279* (2025).

[7] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, et al. 2026. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits. *arXiv preprint arXiv:2603.19173* (2026).

[8] OpenAI. 2025. *Codex CLI.* https://github.com/openai/codex AI coding agent.

[9] Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517* (2025).

[10] Timo Saemann. 2026. KernelBench Precision Search. https://github.com/TimoSaemann/precision-search-kernelbench.

[11] Dipankar Sarkar. 2026. The Correctness Illusion in LLM-Generated GPU Kernels. *arXiv preprint arXiv:2606.20128* (2026).

[12] Mark Saroufim, Jiannan Wang, Bert Maher, Sahan Paliskara, Laura Wang, Shahin Sefati, and Manuel Candales. 2025. *BackendBench: An Evaluation Suite for Testing How Well LLMs and Humans Can Write PyTorch Backends.* https://github.com/meta-pytorch/BackendBench

[13] Ali Tehrani, Yahya Emara, Essam Wissam, Wojciech Paluch, Waleed Atallah, Mohamed S Abdelfattah, et al. 2026. Fine-Tuning GPT-5 for GPU Kernel Generation. *arXiv preprint arXiv:2602.11000* (2026).

[14] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[15] Lingyun Yang, Yuxiao Wang, Shenghao Liang, Linfeng Yang, Daocheng Ying, Chunbo You, Rui Zhang, Luping Wang, Yinghao Yu, Guodong Yang, et al. 2026. Are LLM-Generated GPU Kernels Production-Ready? A Trace-Driven Benchmark and Optimization Agent. *arXiv preprint arXiv:2607.14541* (2026).

[16] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, et al. 2026. Towards automated kernel generation in the era of LLMs. *arXiv preprint arXiv:2601.15727* (2026).

[17] Alex L Zhang, Matej Sirovatka, Erik Schultheis, Benjamin Horowitz, and Mark Saroufim. 2025. KernelBot: A competition platform for writing heterogeneous GPU code. In *Championing Open-source Development in ML Workshop at ICML25.*

[18] Yunxiang Zhang, Ping Yu, Jianyu Wang, Julian Reed, Azalia Mirhoseini, Will Su, et al. 2026. KernelBench-Verified: Do LLM-Generated Kernels Actually Beat PyTorch? *arXiv preprint arXiv:2607.16241* (2026).